Exercise 1 (4+4+4+4 = 16 pts, 15 minutes)

What is the output of the following programs?

(a)

```
void main( ) {
   int array[ ] = {10, 20, 30, 40, 50};
   int *p = array;
   *p++;
   *(++p);
   *p = *(p + 1) + *(p + 2);

   for (int i = 0; i < 5; i++) {
      printf("%d ", array[i]);
   }
}
```

(b)

```
void mystery(int *arr, int start, int end) {
   if (start >= end)
      return;
   int a = arr[start];
   arr[start] = arr[end];
   arr[end] = a;
   mystery (arr, start + 1, end - 1);
}

void main() {
   int arr[ ] = {1, 2, 3, 4, 5};
   mystery (arr, 0, 4);
   for (int i = 0; i < 5; i++) {
      printf("%d ", arr[i]);
   }
}
```

(c)

```
void main( ) {
   int arr[ ] = {1, 2, 3, 4, 5};
   int *p = arr + 2;

   *(p + 2) += 10;
   printf("%d", arr[2]);
}
```

(d)

```
void main( ) {
   int values[ ] = {5, 10, 15, 20};
   int r;
   int *ptr = values + 1;

   r = (*(--ptr)) * (*(ptr + 3)) - (*(ptr++)) + (*(++ptr)) * (*(ptr + 1));

   printf("%d", r);
}
```

Exercise 2 (10 + 10 + 10 = 30 pts, 20 minutes)

In this exercise, you will work with a large 2D matrix of integers stored in a binary file. Due to its size, this matrix cannot be loaded into memory. To solve the problem, we want to write the tasks to work directly on the file. You will write three separate C functions to handle the following tasks:

1. **Store the Matrix**: Write a function that takes as input a 2D matrix of integers with the associated number of rows and columns and stores the matrix into a binary file named “mat.dat”. Think to store the number of rows and columns at the beginning of the file.
2. **Find Maximum Value and Its Position**: Write a function that prints the maximal value and its position (i,j) of the stored matrix. To do so, you should traverse the binary file named “mat.dat” to find the maximal value of the matrix, as well as its position (row and column indices). Remember, you cannot load the matrix into a variable.
3. **Update Value at a Given Position**: Implement a function to update the matrix's value at a specific position (i, j) with a new value. Since the matrix is stored in a binary file, you'll need to perform this operation by directly modifying the appropriate position in the file without loading the matrix. Do not forget to use fseek to navigate directly to the position i,j.

Exercise 3 (15 + 15 = 30 pts, 30 minutes)

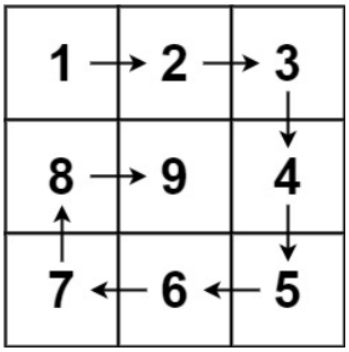
You are given a **sorted** linked list of characters. Each node in the linked list contains a character and a pointer on the next node. Your task is to write a C function that removes the duplications from this list. **The list of characters should not be traversed more than once and you should not use any auxiliary data structure.**

- If the input linked list is: 'a' -> 'a' -> 'b' -> 'c' -> 'd' -> 'd' -> 'd' -> NULL
- The same linked list becomes: 'a' -> 'b' -> 'c'-> 'd' -> NULL

a. Write an iterative function to remove duplicates from a linked list.
b. Write a recursive function to do the same task.

Exercise 4 (20 pts, 25 minutes)

Given a positive integer n, write a function that creates a n x n matrix filled with elements from 1 to n^2 in spiral order.




```
Input: n = 3
Output: [[1,2,3],[8,9,4],[7,6,5]]
```